

# 目 录

|                                            |    |
|--------------------------------------------|----|
| 1. 语法分析测试 .....                            | 2  |
| 1.1. 合法用例 (parser_valid/, 共 8 个) .....     | 2  |
| 1.1.1. pv01_minimal .....                  | 2  |
| 1.1.2. pv02_assign .....                   | 2  |
| 1.1.3. pv03_if_else .....                  | 3  |
| 1.1.4. pv04_while .....                    | 4  |
| 1.1.5. pv05_for_to .....                   | 4  |
| 1.1.6. pv06_for_downto .....               | 5  |
| 1.1.7. pv07_proc_func .....                | 6  |
| 1.1.8. pv08_array .....                    | 7  |
| 1.2. 非法用例 (parser_invalid/, 共 8 个) .....   | 8  |
| 1.2.1. pi01_missing_dot .....              | 8  |
| 1.2.2. pi02_missing_decl_semicolon .....   | 9  |
| 1.2.3. pi03_unmatched_begin_end .....      | 9  |
| 1.2.4. pi04_if_missing_then .....          | 9  |
| 1.2.5. pi05_for_missing_do .....           | 10 |
| 1.2.6. pi06_array_missing_range_op .....   | 10 |
| 1.2.7. pi07_proc_param_missing_colon ..... | 11 |
| 1.2.8. pi08_call_missing_rparen .....      | 11 |
| 1.3. 测试结果汇总 .....                          | 11 |

# 语法分析单元测试报告

## 1. 语法分析测试

所有语法测试均使用以下命令执行（--parse 模式仅做词法和语法分析，输出 AST 结构后停止）：

```
01 ./build/pascc -i <file.pas> --parse
```

合法用例预期输出 Parse succeeded. 并打印 AST 树形结构；非法用例预期输出 Parse error 及错误位置信息。

### 1.1 合法用例（parser\_valid/, 共 8 个）

#### 1.1.1 pv01\_minimal

最小合法程序，验证解析器对空程序体的处理。

源码：

```
01 program pv01;  
02 begin  
03 end.
```

AST 输出：

```
01 Parse succeeded.  
02 Program  
03   Block  
04     List  
05     List  
06     List  
07     List  
08     CompoundStmt
```

所有的 List 节点（常量声明、类型声明、变量声明、子程序声明列表）均为空，CompoundStmt 为空的 begin-end 语句块。AST 结构完整，层级正确。

#### 1.1.2 pv02\_assign

变量声明与基本赋值语句。

源码：

```
01 program pv02;  
02 var a: integer;  
03 begin
```

```
04  a := 1
05  end.
```

**AST 输出:**

```
01 Parse succeeded.
02 Program
03   Block
04     List
05     List
06     List
07       VarDecl
08         List
09           Identifier (a)
10           Identifier (integer)
11     List
12     CompoundStmt
13       AssignStmt
14         Identifier (a)
15         Literal
```

VarDecl 节点包含变量名列表 (Identifier (a)) 和类型标识 (integer)。

AssignStmt 节点的左值为 Identifier, 右值为字面量 Literal。

### 1.1.3 pv03\_if\_else

if-then-else 条件语句, 验证悬挂 else 消歧。

**源码:**

```
01 program pv03;
02 var a: integer;
03 begin
04   if a = 0 then
05     a := 1
06   else
07     a := 2
08 end.
```

**AST 输出:**

```
01 Parse succeeded.
02 Program
03   Block
04     ...
05     CompoundStmt
06       IfStmt
07         BinaryExpr
08         Identifier (a)
```

```

09      Literal
10      AssignStmt
11      Identifier (a)
12      Literal
13      AssignStmt
14      Identifier (a)
15      Literal

```

IfStmt 节点包含三个子节点：条件表达式 BinaryExpr (a = 0)、then 分支赋值、else 分支赋值。else 正确绑定到最近的 if。

#### 1.1.4 pv04\_while

while-do 循环语句。

源码：

```

01 program pv04;
02 var a: integer;
03 begin
04   while a < 10 do
05     a := a + 1
06 end.

```

AST 输出：

```

01 Parse succeeded.
02 Program
03   Block
04     ...
05     CompoundStmt
06       WhileStmt
07         BinaryExpr
08           Identifier (a)
09           Literal
10         AssignStmt
11           Identifier (a)
12           BinaryExpr
13             Identifier (a)
14             Literal

```

WhileStmt 包含条件表达式 BinaryExpr (a < 10) 和循环体 AssignStmt。循环体内的二元表达式 BinaryExpr (a + 1) 嵌套正确。

#### 1.1.5 pv05\_for\_to

for-to 递增循环语句。

源码：

```

01 program pv05;
02 var i: integer;
03 begin
04   for i := 1 to 10 do
05     i := i + 1
06 end.

```

**AST 输出:**

```

01 Parse succeeded.
02 Program
03   Block
04     ...
05     CompoundStmt
06       ForStmt
07         Identifier (i)
08         Literal
09         Literal
10         AssignStmt
11           Identifier (i)
12           BinaryExpr
13             Identifier (i)
14             Literal

```

ForStmt 包含循环变量 Identifier (i)、起始值 Literal、终止值 Literal 和循环体。第四个参数为 false（递增方向，to 非 downto）。

### 1.1.6 pv06\_for\_downto

for-downto 递减循环语句。

**源码:**

```

01 program pv06;
02 var i: integer;
03 begin
04   for i := 10 downto 1 do
05     i := i - 1
06 end.

```

**AST 输出:**

```

01 Parse succeeded.
02 Program
03   Block
04     ...
05     CompoundStmt
06       ForStmt
07         Identifier (i)

```

```

08      Literal
09      Literal
10      AssignStmt
11          Identifier (i)
12      BinaryExpr
13          Identifier (i)
14      Literal

```

ForStmt 的第四个参数为 true（递减方向，downto），与 pv05 形成正交对照。

### 1.1.7 pv07\_proc\_func

过程与函数声明，参数传递和调用。

源码：

```

01 program pv07;
02
03 procedure p(x: integer);
04 begin
05     x := x + 1
06 end;
07
08 function f(y: integer): integer;
09 begin
10     f := y
11 end;
12
13 begin
14     p(1);
15     write(f(2))
16 end.

```

AST 输出：

```

01 Parse succeeded.
02 Program
03   Block
04     List
05     List
06     List
07     List
08       ProcDecl
09       List
10       ParamDecl
11       List
12         Identifier (x)
13         Identifier (integer)

```

```

14      Block
15      List
16      List
17      CompoundStmt
18      AssignStmt
19      Identifier (x)
20      BinaryExpr
21      Identifier (x)
22      Literal
23  FuncDecl
24      List
25      ParamDecl
26      List
27      Identifier (y)
28      Identifier (integer)
29      Block
30      List
31      List
32      CompoundStmt
33      AssignStmt
34      Identifier (f)
35      Identifier (y)
36  CompoundStmt
37      ProcCall
38      Literal
39      ProcCall
40      ProcCall
41      Literal

```

ProcDecl 和 FuncDecl 各有参数列表、返回类型和体 block。过程调用 p(1) 和函数调用 f(2) 均表示为 ProcCall 节点，嵌套调用 write(f(2)) 正确构建了 ProcCall → ProcCall → Literal 的嵌套 AST。

### 1.1.8 pv08\_array

数组声明与下标访问。

源码：

```

01 program pv08;
02 var a: array[1..10] of integer;
03     i: integer;
04 begin
05     a[1] := 2;
06     i := a[1]
07 end.

```

AST 输出：

```

01 Parse succeeded.
02 Program
03   Block
04     ...
05       VarDecl
06         List
07           Identifier (a)
08         ArrayType
09           Literal
10           Literal
11           Identifier (integer)
12       VarDecl
13         List
14           Identifier (i)
15           Identifier (integer)
16     ...
17   CompoundStmt
18     AssignStmt
19       ArrayAccess
20         Identifier (a)
21         Literal
22         Literal
23     AssignStmt
24       Identifier (i)
25       ArrayAccess
26         Identifier (a)
27         Literal

```

ArrayType 节点包含下界 Literal、上界 Literal 和元素类型 Identifier (integer)。ArrayAccess 节点在赋值左值（写入 a[1]）和右值（读取 a[1]）位置上均正确构建。

## 1.2 非法用例 (parser\_invalid/, 共 8 个)

### 1.2.1 pi01\_missing\_dot

程序末尾缺少句点。

源码:

```

01 program pi01;
02 begin
03 end

```

错误输出:

```

01 Parse error at 3:4: syntax error
02 Parsing failed.

```



解析器在文件末尾检测到缺少 .，报错位置精确指向第 3 行第 4 列（end 之后）。

### 1.2.2 pi02\_missing\_decl\_semicolon

变量声明之间缺少分号。

源码：

```
01 program pi02;  
02 var a: integer  
03     b: integer;  
04 begin  
05     a := 1  
06 end.
```

错误输出：

```
01 Parse error at 3:5 near 'b': syntax error  
02 Parsing failed.
```

a: integer 后缺少分号，解析器在看到下一行开头的 b 时报错，行列信息准确。

### 1.2.3 pi03\_unmatched\_begin\_end

begin/end 不配对。

源码：

```
01 program pi03;  
02 begin  
03     begin  
04         write(1)  
05 end.
```

错误输出：

```
01 Parse error at 5:4 near '.': syntax error  
02 Parsing failed.
```

内层 begin 缺少对应的 end，解析器在遇到程序结尾的 . 时检测到配对不匹配。

### 1.2.4 pi04\_if\_missing\_then

if 语句缺少 then 关键字。

源码：

```
01 program pi04;
02 var a: integer;
03 begin
04   if a = 1
05     a := 2
06 end.
```

错误输出:

```
01 Parse error at 5:5 near 'a': syntax error
02 Parsing failed.
```

if a = 1 后缺少必需的 then，解析器在看到赋值左值 a 时报错。

### 1.2.5 pi05\_for\_missing\_do

for 循环缺少 do 关键字。

源码:

```
01 program pi05;
02 var i: integer;
03 begin
04   for i := 1 to 10
05     i := i + 1
06 end.
```

错误输出:

```
01 Parse error at 5:5 near 'i': syntax error
02 Parsing failed.
```

for i := 1 to 10 后缺少 do，解析器在看到循环体中的 i 时报错。

### 1.2.6 pi06\_array\_missing\_range\_op

数组声明中范围操作符错误 (， 替代 ..)。

源码:

```
01 program pi06;
02 var a: array[1.10] of integer;
03 begin
04   a[1] := 2
05 end.
```

错误输出:

```
01 Parse error at 2:18 near ']': syntax error
02 Parsing failed.
```

1.10 被解析为一个实数而非范围 1..10，解析器在遇到 ] 时发现语法不匹配。

### 1.2.7 pi07\_proc\_param\_missing\_colon

过程参数声明缺少冒号。

源码：

```
01 program pi07;  
02 procedure p(x integer);  
03 begin  
04 end;  
05 begin  
06 end.
```

错误输出：

```
01 Parse error at 2:15 near 'integer': syntax error  
02 Parsing failed.
```

x 和 integer 之间缺少 :，解析器在看到 integer 关键字时报错。

### 1.2.8 pi08\_call\_missing\_rparen

过程调用缺少右括号。

源码：

```
01 program pi08;  
02 begin  
03   write(1  
04 end.
```

错误输出：

```
01 Parse error at 4:1 near 'end': syntax error  
02 Parsing failed.
```

write(1 缺少 )，解析器在遇到下一行的 end 时报错。

## 1.3 测试结果汇总

| 用例                            | 类型 | 结果        |
|-------------------------------|----|-----------|
| pv01_minimal                  | 正例 | AST 构建正确  |
| pv02_assign                   | 正例 | AST 构建正确  |
| pv03_if_else                  | 正例 | AST 构建正确  |
| pv04_while                    | 正例 | AST 构建正确  |
| pv05_for_to                   | 正例 | AST 构建正确  |
| pv06_for_downto               | 正例 | AST 构建正确  |
| pv07_proc_func                | 正例 | AST 构建正确  |
| pv08_array                    | 正例 | AST 构建正确  |
| pi01_missing_dot              | 负例 | 正确检测到语法错误 |
| pi02_missing_decl_semicolon   | 负例 | 正确检测到语法错误 |
| pi03_unmatched_begin_end      | 负例 | 正确检测到语法错误 |
| pi04_if_missing_then          | 负例 | 正确检测到语法错误 |
| pi05_for_missing_do           | 负例 | 正确检测到语法错误 |
| pi06_array_missing_range_op   | 负例 | 正确检测到语法错误 |
| pi07_proc_param_missing_colon | 负例 | 正确检测到语法错误 |
| pi08_call_missing_rparen      | 负例 | 正确检测到语法错误 |

语法分析测试结果汇总（16 个用例全部通过）